AVL Trees

Recall the operations (e.g. `find`, `insert`, `delete`) of a binary search tree. The runtime of these operations were all $O(h)$ where h represents the height of the tree, defined as the length of the longest branch. In the worst case, all the nodes of a tree could be on the same branch. In this case, $h = n$, so the runtime of these binary search tree operations are $O(n)$. However, we can maintain a much better upper bound on the height of the tree if we make efforts to balance the tree and even out the length of all branches. An AVL trees is a binary search tree that balances itself every time an element is inserted or deleted. In addition to the invariants of a BST, **each node of an AVL tree has the invariant property that the heights of the sub-tree rooted at its children differ by at most one**, i.e.:

$$|\texttt{height(node.left)} - \texttt{height(node.right)}| \leq 1 \tag{1}$$

Height Augmentation

In AVL trees, we augment each node to keep track of the node's height.

```
def height(node):
    if node is None:
        return -1
    else:
        return node.height

def update_height(node):
    node.height = max(height(node.left), height(node.right)) + 1
```

Every time we insert or delete a node, we need to update the height all the way up the ancestry until the height of a node doesn't change.

AVL Insertion, Deletion and Rebalance

We can insert a node into or delete a node from a AVL tree like we do in a BST. But after this, the height invariant (1) of the AVL tree may not be satisfied any more.

For insertion, there are 2 cases where the invariant will be violated:

1. The left child of node x is heavier than the right child. Inserting into the left child may imbalance the AVL tree.

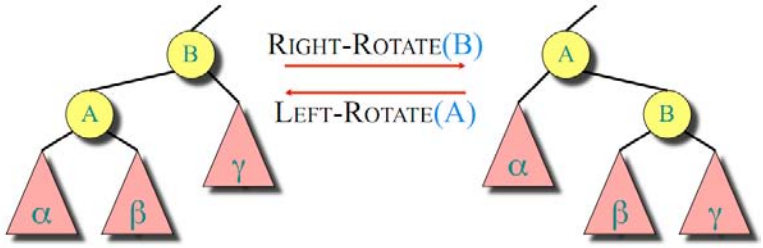
2. The right child of node x is heavier than the left child. Inserting into the right child may imbalance the AVL tree.

For deletion, the cases are analogous.

So we need to reblance the tree to maintain the invariant, starting from the node inserted or the parent of the deleted node, and continue up.

There are two operations needed to help balance an AVL tree: a left rotation and a right rotation. Rotations simply re-arrange the nodes of a tree to shift around the heights while maintaining the

order of its elements. Making a rotation requires re-assigning `left`, `right`, and `parent` of a few nodes, and **updating their heights**, but nothing more than that. Rotations are $O(1)$ time operations.




```
def rebalance(self, node):
    while node is not None:
        update_height(node)
        if height(node.left) >= 2 + height(node.right):
            if height(node.left.left) >= height(node.left.right):
                self.right_rotate(node)
            else:
                self.left_rotate(node.left)
                self.right_rotate(node)
        elif height(node.right) >= 2 + height(node.left):
            if height(node.right.right) >= height(node.right.left):
                self.left_rotate(node)
            else:
                self.right_rotate(node.right)
                self.left_rotate(node)
        node = node.parent
```

Note that `rebalance` includes `upate_height` as well.

MIT OpenCourseWare
http://ocw.mit.edu

6.006 Introduction to Algorithms
Fall 2011

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.